

COM2002-1 Programación de Software de Sistemas**Profesor :** Joaquín Soto**Ayudante:** Arnaldo González

Pauta - Ayudantía Repaso Control 1

Soluciones esperadas y criterios de corrección

P1. Preguntas conceptuales (4 puntos).

- a) **(1 punto)** 'char *p' declara un puntero que almacena una dirección de memoria; 'char arreglo[20]' reserva espacio para 20 caracteres dentro de ese contexto. Para un arreglo declarado localmente, 'sizeof(arreglo)' entrega el tamaño total del arreglo (20 bytes), mientras que 'sizeof(p)' entrega el tamaño del puntero, no el tamaño de la cadena apuntada. Se espera mencionar que el resultado de 'sizeof(p)' depende de la arquitectura, pero no corresponde a 20 por el solo hecho de apuntar a un arreglo.
- b) **(1 punto)** Con 'const char *s' no se deben modificar los caracteres a través de 's'. Sí se puede cambiar el propio puntero si fue declarado sin 'const' a la izquierda del '*'. Ejemplo válido: 'printf("%c", s[0])'. Ejemplo inválido: 's[0] = 'A''.
- c) **(1 punto)** La inversión puede hacerse intercambiando elementos desde los extremos hacia el centro, por lo que no se necesita un segundo arreglo completo. Una solución correcta tiene complejidad temporal $O(n)$ y espacio auxiliar $O(1)$, aparte del arreglo de entrada.
- d) **(1 punto)** La condición base es que el nodo sea una hoja. En ese caso se retorna directamente el valor almacenado en esa hoja. Si el nodo es interno, se evalúan recursivamente sus hijos y luego se aplica la operación almacenada en el nodo.

P2. Unir cadenas (4 puntos).

Una solución completamente válida es:

```
#include <stdlib.h>
#include <string.h>

char *unir_cadenas(const char *a, const char *b) {
    size_t la = strlen(a);
    size_t lb = strlen(b);

    char *resultado = malloc((la + lb + 1) * sizeof(char));
    if (resultado == NULL) {
        return NULL;
    }

    for (size_t i = 0; i < la; i++) {
        resultado[i] = a[i];
    }
```

```
for (size_t j = 0; j < lb; j++) {
    resultado[la + j] = b[j];
}

resultado[la + lb] = '\0';
return resultado;
}
```

Criterio de corrección sugerido: 1 punto por calcular correctamente el tamaño incluyendo el '\0'; 1 punto por reservar y verificar la memoria; 1 punto por copiar las dos cadenas en el orden correcto; 1 punto por terminar correctamente el string y retornar la dirección reservada. También se acepta una implementación equivalente usando 'strcpy' y 'strcat', siempre que la memoria se reserve correctamente y ninguna cadena original sea modificada.

P3. Rotación en el mismo arreglo (3 puntos).

Una forma sencilla es desplazar todos los caracteres una posición a la izquierda y guardar temporalmente el primer carácter:

```
void rotar_izquierda(char *s) {
    if (s == NULL || s[0] == '\0') {
        return;
    }

    char primero = s[0];
    int i = 0;

    while (s[i + 1] != '\0') {
        s[i] = s[i + 1];
        i++;
    }

    s[i] = primero;
}
```

Otra respuesta válida puede utilizar reversas parciales o intercambios equivalentes, siempre que el resultado sea correcto y no se cree otro arreglo completo. La solución mostrada tiene tiempo $O(n)$ y espacio auxiliar $O(1)$.

Criterio de corrección sugerido: 1 punto por recorrer correctamente hasta el terminador; 1 punto por desplazar sin perder el primer carácter; 1 punto por conservar la terminación '\0' y respetar la restricción de memoria.

P4. Estructuras en C: concepto y uso (8 puntos).

- a) **(1 punto)** Una 'struct' permite agrupar distintas variables relacionadas bajo una misma entidad. Por ejemplo, una persona puede tener nombre, edad y promedio almacenados dentro de una sola estructura, en lugar de manejar cada dato como variable independiente.
- b) **(1 punto)** Una declaración válida es:

```
Persona p;  
p.edad = 21;
```

La variable 'p' es una instancia de la estructura definida mediante 'typedef'. Se accede a sus campos usando el operador '.'.

- c) **(1 punto)** El operador '.' se utiliza cuando se tiene directamente una variable de tipo estructura. El operador '->' se utiliza cuando se tiene un puntero a una estructura. Por ejemplo:

```
Persona p;  
Persona *ptr = &p;  
  
p.edad = 21;  
ptr->edad = 22;
```

Ambas instrucciones acceden al mismo campo, pero a través de variables de distinta naturaleza.

- d) **(1 punto)** Pasar un puntero permite trabajar sobre la estructura original sin copiarla completa. Además, una función puede modificar directamente sus campos. Por ejemplo:

```
void aumentar_edad(Persona *p) {  
    p->edad++;  
}
```

También es especialmente útil cuando la estructura contiene muchos datos, ya que se evita una copia completa.

- e) **(1 punto)** Una estructura 'Producto' podría contener, por ejemplo, un código identificador, un precio y la cantidad disponible en stock. Una función podría recibir 'Producto *p' para modificar el stock después de una venta, por ejemplo:

```
void descontar_stock(Producto *p, int cantidad);
```

No es necesario implementar la función; lo importante es reconocer que la estructura agrupa los datos relacionados y que el puntero permite modificarlos desde una función.

- f) **(1 punto)** Una forma válida de crear e inicializar 'p' es:

```
Punto p = {3.5, -1.0};
```

También se acepta una inicialización equivalente usando nombres de campos, siempre que 'x' e 'y' queden correctamente asignados.

- g) **(1 punto)** 'Persona p' representa una sola instancia de la estructura, mientras que 'Persona personas[50]' representa un arreglo con 50 elementos de tipo 'Persona'. El arreglo sería útil, por ejemplo, para almacenar los datos de todos los estudiantes de un curso.
- h) **(1 punto)** Si la función solamente necesita leer los datos, ambas alternativas pueden ser correctas. Con 'Persona p' se pasa una copia de la estructura. Con 'Persona *p' se pasa su dirección y la función accede a los campos mediante '->'. Por ejemplo, al usar la segunda opción una llamada podría ser:

```
Persona p;  
mostrar_persona(&p);
```

Si se busca evitar copiar una estructura grande, pasar un puntero es una alternativa conveniente; además, puede declararse como 'const Persona *p' si se quiere dejar explícito que la función no debe modificarla.

Criterio de corrección sugerido: 1 punto por cada apartado correcto. En los apartados de ejemplo, se aceptan nombres de campos y situaciones equivalentes siempre que respeten la idea central. No se exige una implementación extensa.

Idea clave para el control: saber moverse entre arreglos y punteros, recordar que un string termina en '\0', distinguir memoria automática de memoria dinámica, y comprender cómo definir y utilizar 'struct, así como cuándo conviene usar punteros a estructuras.